CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical-1

Practical 1: Array Operations: Traversal, Rotation, and Frequency Analysis

Problem 1

A bakery prepares n items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours h, print the final display order.

Describe the approach you used. How does it behave when h is very large, say 10 million? Is there a way to get the correct result without performing the operation h times?

Code:-

https://github.com/smitmalaviya80-sys/FDSA-Batch-A/tree/main/practical-1

Output:-

```
PS C:\Users\PC\Desktop\FDSA> cd "c:\Users\PC\Desktop\FDSA\practical-1\"; g++ "bakreysys.cpp" -o "bakr
eysys.exe"; if ($?) { & ".\bakreysys.exe" }
Enter the no. of Items :5
Enter no. of Hour : 4
3
2
5
7
4
4 3 2 5 7
PS C:\Users\PC\Desktop\FDSA\practical-1>
```

Problem-faced:-

Difficulty I show Regarding Problem Solving

In this problem I one the I want to learn optimal approach to solve this problem here I used two for loop I want learn how to solve without using second loop within solve only 1 loop that is learn in this problem .\

Problem 2

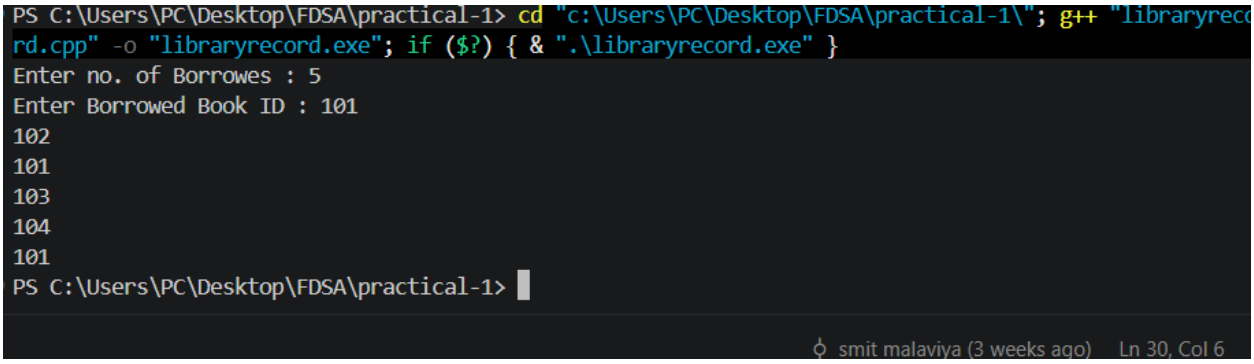
A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs.

Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of a way that requires fewer passes?

Code:-

https://github.com/smitmalaviya80-sys/FDSA-Batch-A/tree/main/practical-1

Output



Problem-faced:-

Here is I show some difficulty how to check each and every element 1 time we have check 0 to i then I check i+1 to n after this problem I have been solved.

Problem 3

A school newspaper editor wants to highlight the most impressive word from a submitted article

on the front page. The word is chosen simply by length — the longest one wins. Given a

sentence, print the winning word and how many letters it has.

Describe the approach you used. What does your solution do when two words have the same

length? Is that behavior intentional, and can you think of a way to handle it explicitly?

Code:-

https://github.com/smitmalaviya80-sys/FDSA-Batch-A/tree/main/practical-1

Output:-

```
PS C:\Users\PC\Desktop\FDSA\practical-1> cd "c:\Users\PC\Desktop\FDSA\practical-1\"; g++ "newsed
cpp" -o "newseditor.exe"; if ($?) { & ".\newseditor.exe" }
Enter TEXT : smit malaviya
longestest Word: malaviya
length Of the word : 8
PS C:\Users\PC\Desktop\FDSA\practical-1> S
```

Problem-faced:-

Here this problem I have been solved by using string library then I peak empty string they store temporary then empty string will be stored null character them find max length string and number string length.

Conclusion:-

This lab demonstrated three core DSA concepts: optimizing repeated operations using cyclic patterns (Problem 1), efficient frequency counting with hash maps for single-pass duplicate detection (Problem 2), and linear search with proper input preprocessing (Problem 3). Together, they show that recognizing patterns and choosing the right data structure often matters more than brute-force computation for writing efficient, scalable solutions.

Key-Question:-

1. [Problem 1] Why the order repeats: The row has only n possible rotated arrangements, and rotation is deterministic, so by the pigeonhole principle the sequence must eventually revisit a state — creating a cycle. The original order reappears exactly when h is a multiple of n, so the cycle length is n.

2. 2. [Problem 2] Book ID appearing three times: It should appear once in the output, since we're identifying which IDs need restocking, not counting repeats. This works correctly if a frequency map is used and output comes from keys with count > 1; verify by testing an ID appearing 3 times and confirming it prints only once.

3. 3. [Problem 3] Ignoring attached punctuation: Punctuation must be stripped from each word after splitting but before measuring length, since raw splitting on whitespace leaves symbols attached, inflating the count. The core length-comparison logic stays the same — only the token needs cleaning first.

Prcatical-2

Practical Aim

To implement linear and binary search using both iterative and recursive approaches.

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where

the function calls itself to continue checking — and report the position of the target plate if found.

Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

Code:-

https://github.com/smitmalaviya80-sys/FDSA-Batch-A/tree/main/practical-2

Output:-

```
PS C:\Users\PC\Desktop\FDSA> cd practical-2
PS C:\Users\PC\Desktop\FDSA\practical-2> cd "c:\Users\PC\Desktop\FDSA\practical-2\"; g++ "parkinglots
.cpp" -o "parkinglots.exe"; if ($?) { & ".\parkinglots.exe" }
Iterative index 2
Recursive index 2
PS C:\Users\PC\Desktop\FDSA\practical-2>
```

Problem-faced:

Here I show first how break when leave the middle of checking after helper arrive hoe fetch already continues loop that is biggest error I have occurred but it error I have been solved with using recusive function they return same function but next index value they will be taken.

Problem-2:

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code. Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

Code:-

https://github.com/smitmalaviya80-sys/FDSA-Batch-A/tree/main/practical-2

Output:-

```
PS C:\Users\PC\Desktop\FDSA\practical-2> cd "c:\Users
intains.cpp" -o "librarianmaintains.exe"; if ($?) { &
Enter the number of elements: 5
Enter 5 sorted elements:
20
30
42
78
99
Enter element to search: 78
Element  index 3
PS C:\Users\PC\Desktop\FDSA\practical-2>
```

Problem-faced:

While implementing this problem, I faced challenges in correctly updating the left and right boundaries after comparing the middle element with the target. I also had to write the proper base case in the recursive approach to stop the search when the target was not found. This approach works correctly only if the list is sorted. If the list is not sorted, the search may discard the part containing the target and produce an incorrect result or fail to find the target even when it exists.

Conclusion:-

This lab compared linear and binary search approaches, showing that sorted data enables far more efficient searching (O(log n) vs O(n)) but only when the sortedness condition truly holds. It also demonstrated how exploiting structural properties — like row/column order in a grid — reduces search cost from $O(n^2)$ to O(n), reinforcing that data structure and precondition checks are as important as the algorithm itself.

Key Question

1. [Problem 1] Skipping plates entirely: Linear search can only skip elements if the list has structure, like being sorted. Problem 2 shows that sorted data allows binary search to eliminate half the remaining elements each step instead of checking one by one.

2. [Problem 2] Binary search on unsorted data: It would not always be wrong — results depend on chance, since the elimination logic wrongly assumes an order that isn't actually there, making outcomes unreliable rather than consistently incorrect.

Practical-3

Practical Aim

To implement and compare multiple sorting algorithms and develop the ability to reason about

which sorting strategy suits a given situation.

PR-3.1

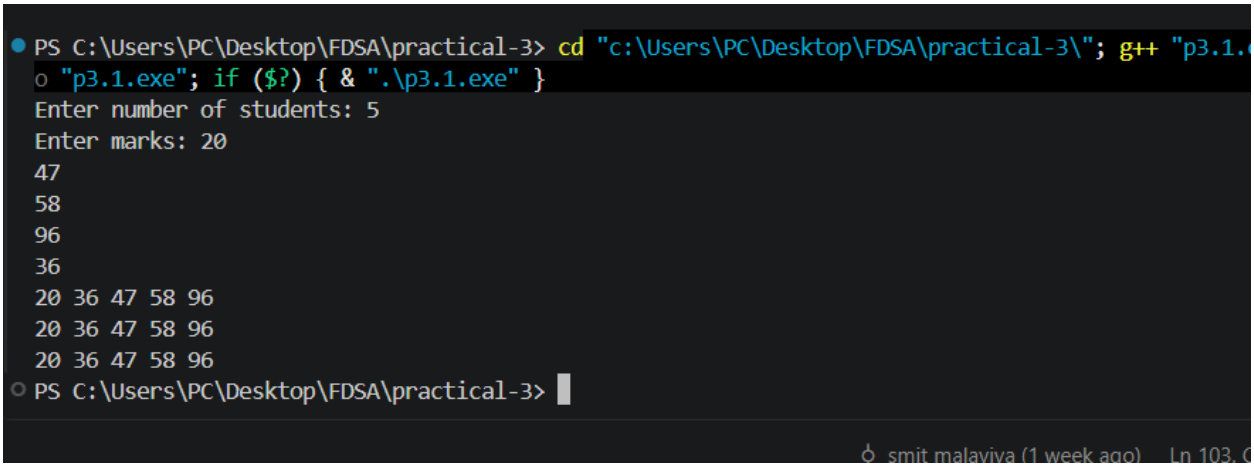
A teacher has a stack of student answer sheets with marks written on them and needs to

arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks. Describe how each of the three methods works in your own words. If the sheets were already in order before the teacher started, which method would finish the fastest and why?

Code:-

https://github.com/smitmalaviya80-sys/FDSA-Batch-A/tree/main/practical-3

output:-



Problem-faced:-

In this practical I have been learned diff types of shorting algorithm like bubble short , insertion , selection short 1. In the bubble short compare two adjustment element and swap them according condition , 2. In insertion short first take index 0 element as a shorted array and another element compare their previous element until if it previous element smaller than current element 3. Selection short first find min element and add the front index of array.

Key-Question:-

1. [Problem 1] Why the order repeats: The row has only n possible rotated arrangements, and rotation is deterministic, so by the pigeonhole principle the sequence must

eventually revisit a state — creating a cycle. The original order reappears exactly when h is a multiple of n, so the cycle length is n.

2. 2. [Problem 2] Book ID appearing three times: It should appear once in the output, since we're identifying which IDs need restocking, not counting repeats. This works correctly if a frequency map is used and output comes from keys with count > 1; verify by testing an ID appearing 3 times and confirming it prints only once.

3. 3. [Problem 3] Ignoring attached punctuation: Punctuation must be stripped from each word after splitting but before measuring length, since raw splitting on whitespace leaves symbols attached, inflating the count. The core length-comparison logic stays the same — only the token needs cleaning first.

PR-3.2

A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are

stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result.

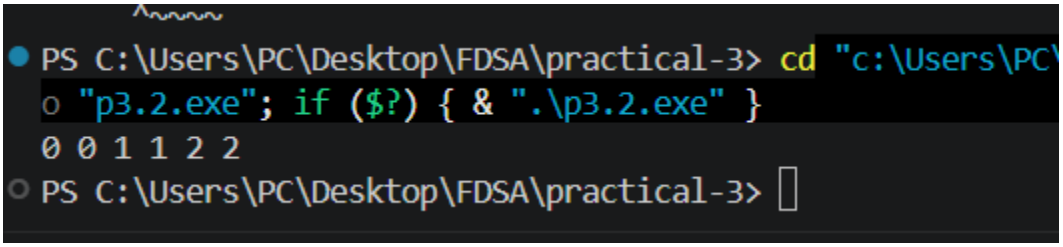
Describe the approach you used. Does your solution make more than one pass through the list?

Can you think of a way to do it in exactly one pass?

Code:-

https://github.com/smitmalaviya80-sys/FDSA-Batch-A/tree/main/practical-3

Output:-



Problem-faced:-

In this practical I have been performed dutch national flag algorithm problem in this problem first find total number of one,two and zero than add array via loop but first add 0 like while(zero--) loop runs until condition false inside loop arr[index]=0 , declare index =0 then when first loop end start next loop same as first …. And of inside array first show like 0,0,0,1,1,2,2

Conclusion:-

This lab implemented and compared bubble, selection, and insertion sort, showing each method's mechanics and that insertion sort performs best (O(n)) on already-sorted input due to its early-exit comparison, unlike bubble and selection sort's fixed passes. The Dutch National Flag problem (Problem 2) further demonstrated how in-place partitioning with three pointers can sort a three-valued array in a single pass without extra storage, reinforcing that algorithm choice should depend on input characteristics and constraints like memory and pass count.

Key Questions

1. [Problem 1] Early sorted-list detection: Bubble sort can detect this using a swapped flag — if no swaps occur in a pass, the list is already sorted and it breaks early. Selection sort always scans fully to find the minimum regardless of order, and insertion sort skips work per element but still visits every index, so neither can fully early-exit like bubble sort.

2. [Problem 2] Eliminating the second pass: The counting step (tallying 0s, 1s, 2s before placing them) forces the second pass, since counts must be known before writing the final array. Using the Dutch National Flag algorithm with three pointers (low, mid, high) swaps each element into place as it's encountered, achieving the sort in exactly one pass.

Practical 4

Practical 4.1

A hospital manages a queue of patient tokens. New critical patients must be added to the front, routine patients are added to the end, and occasionally a patient with a priority number must be inserted at a specific position in the queue. Given a sequence of such operations, implement all three insertion types and print the final queue after each operation.

Code :

https://github.com/smitmalaviya80-sys/FDSA-Batch-A/tree/main/practical-4

Output :

```
1. Critical Patient (Front)
2. Routine Patient (End)
3. Priority Patient (Specific Position)
4. Exit
Enter choice: 1
Enter patient token: 2
Queue: 2

1. Critical Patient (Front)
2. Routine Patient (End)
3. Priority Patient (Specific Position)
4. Exit
Enter choice: 1
Enter patient token: 3
Queue: 3 2

1. Critical Patient (Front)
2. Routine Patient (End)
3. Priority Patient (Specific Position)
4. Exit
Enter choice: 2
Enter patient token: 5
Queue: 3 2 5

1. Critical Patient (Front)
2. Routine Patient (End)
3. Priority Patient (Specific Position)
4. Exit
Enter choice: 3
Enter patient token: 7
Enter position: 1
Queue: 7 3 2 5

1. Critical Patient (Front)
2. Routine Patient (End)
```

Problem Faced :

In this Practical , I faced problem to give next node after creating node and also faced problem to add node at specific point and to link two node after creating new node.

Key Question :

[Problem 1] Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something else entirely? Justify your choice and explain what assumption about the caller your choice makes.

If the position given is greater than the current length of the linked list, the operation is considered invalid. My solution explicitly checks the position and reports an error instead of silently inserting the new patient at the end. This is a better approach because it prevents incorrect input from being hidden and ensures that the requested position is followed correctly. It assumes that the caller is responsible for providing a valid position, and if an invalid position is provided, the program informs the caller rather than causing an error or changing the requested operation.

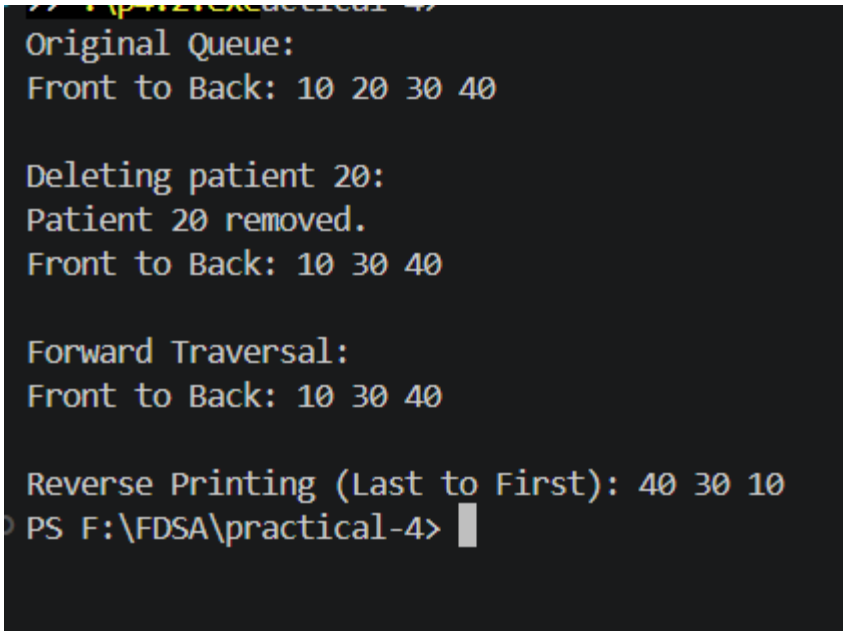
Practical 4.2

The hospital also needs to remove a patient token from the queue when a patient leaves, print all remaining tokens from last to first for an end-of-day audit, and display the full queue from front to back at any point. Implement deletion by value, reverse printing, and forward traversal on the same queue from Problem 4.1a.

Code :

https://github.com/smitmalaviya80-sys/FDSA-Batch-A/tree/main/practical-4

Output :



Problem faced :

In this Practical , I faced problem to give next node after deleting node and also faced problem to link two node after deleting new node.

Key Question :

The hospital also needs to remove a patient token from the queue when a patient leaves, print all remaining tokens from last to first for an end-of-day audit, and display the full queue from front to back at any point. Implement deletion by value, reverse printing, and forward traversal on the same queue from Problem 4.1a.

If the node to be deleted is the only node in the list, only **one pointer, the head pointer, needs to change**, and it must be set to NULL because the list becomes empty. This is structurally different from deleting a node in the middle, where the next pointer of the previous node must be changed to point to the node after the deleted node. In the single-node case, there is no previous or next node, so we simply make head = NULL and delete the node.

Practical-5:

Problem-1

A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation. Describe how your insertion after a given song works. What extra step does a doubly linked list require here that a singly linked list did not? What happens if the given song is not found in the playlist?

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output:

```
o "p5.1.exe"; if ($?) { & ".\p5.1.exe" }
PS C:\Users\PC\Desktop\FDSA\practical-5> cd "c:\Users\
o "p5.1.exe"; if ($?) { & ".\p5.1.exe" }
Song1
Song1 Song2
Song1 Song2 Song3
Song1 Song2 Song4 Song3
Song2 Song4 Song3
Total songs: 3
PS C:\Users\PC\Desktop\FDSA\practical-5>
```

Problem Faced:-

I faced problem on this after first insert that is some time I forgot the step but now I remember a first link next then prev and than prev node link current and next node link with prev(current)

Key question

1. [Problem 1] A doubly linked list maintains two links per node instead of one. When deleting the front node, explain why having a backward link on the second node creates a responsibility that did not exist in a singly linked list — and what goes wrong if you forget to handle it.

➔ When deleting the front node of a doubly linked list, the second node becomes the new head. We must set its prev pointer to NULL, because it should no longer point to the deleted node. In a singly linked list, this step is not required because there is no prev link. If we forget it, the new head may point to a deleted node, causing a dangling pointer and possible program errors or crashes.

Problem-2

A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave.

Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each operation.

Describe how your solution ensures the circle stays unbroken after a student leaves. What is

the one case where breaking the circle is most likely to happen as a bug, and how did you

handle it?

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output:

```
PS C:\Users\PC\Desktop\FDSA> cd "c:\Users\PC\Desk
; if ($?) { & ".\p5.2.exe" }
A
A B
A B C
A C
A
A B
A B C
A C
PS C:\Users\PC\Desktop\FDSA\practical-5>
```

Problem faced:

Also I have faced problem in this code is to joint circular link list and doubley link to join at end to first.

Key question:

2. [Problem 2] If your traversal of the circular list does not stop at the right moment, it will

loop forever. Without referencing any specific code, explain what condition must be true

at the moment you stop, and why that condition would fail if you set the last node's link to

null.

➔ In a circular linked list, traversal should stop when you reach the **starting node again**, because the last node points back to the first node. If the last node's link is set to `NULL`, the circle is broken, so the traversal would stop at `NULL` instead of detecting that it has returned to the starting node.

Practical-6:

To implement stack using array and linked list, and apply stack-based logic to expression conversion, evaluation, and bracket validation problems.

Problem 1:

A cafeteria stacks clean trays on a fixed-size counter. New trays are always placed on top, and

customers always take from the top. The counter can hold at most n trays at a time — if it is full,

no more trays can be added, and if it is empty, no tray can be taken. Given a sequence of place

and take operations, implement this fixed-capacity tray stack and print the current top tray after

each operation. Report an error if a place or take operation cannot be performed.

Describe how your solution tracks whether the stack is full or empty. What are the exact

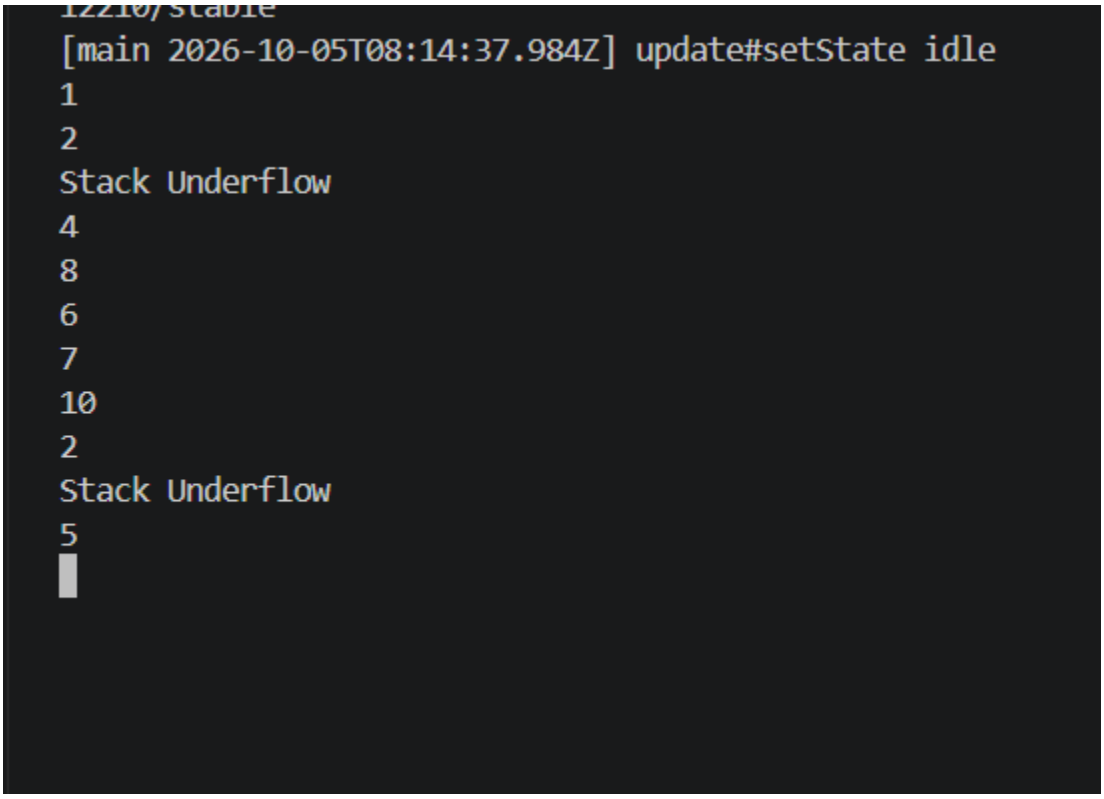
conditions you check before a push and before a pop, and what does your program do when

those conditions fail?

Code :

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output :



Problem faced:

In this practical I faced the problem is the stack implementation and also revice underflow and overflow condition

Key question :

Problem 1: The front and rear pointers should move back to the beginning of the array when they reach the last position, using modulo logic. The array is full when the next position of rear is equal to front, i.e. (rear + 1) % size == front.

Problem -2:

A web browser keeps track of pages visited so that the back button always returns to the most recently visited page. Unlike a fixed counter, the browser has no hard limit on how many pages it can remember — it grows as the user visits more pages and shrinks as they press back.

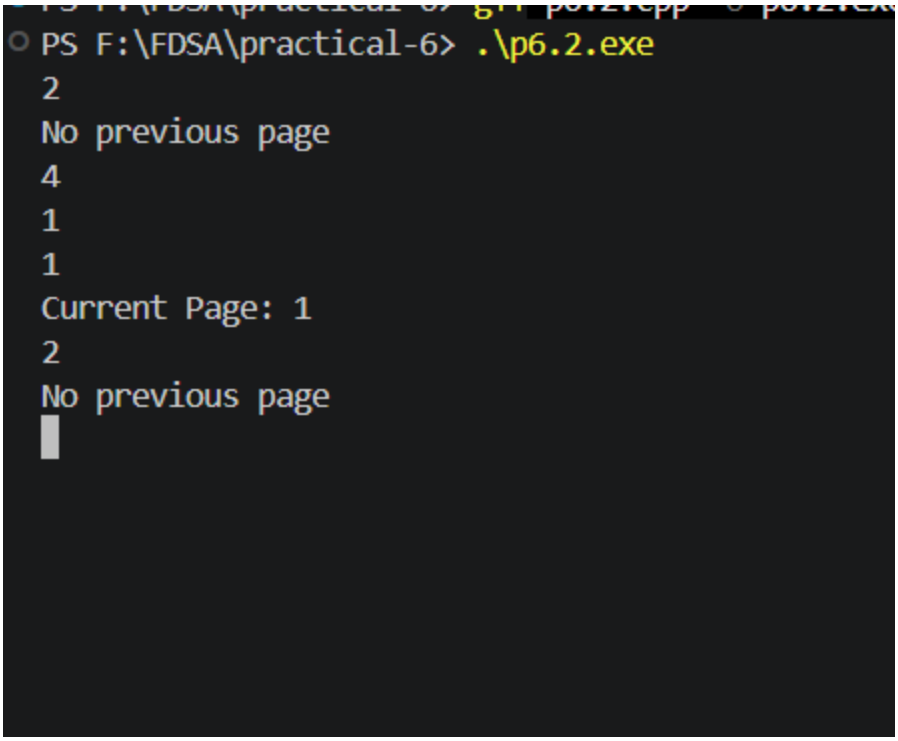
Given a sequence of visit and back operations, implement this unlimited page history and print the current page after each operation.

Describe how your solution differs from Problem 1 in how it handles memory. What happens in your solution when the user presses back on the very first page with no history left?

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output:



Problem faced:

Most important thing is the implement the previous practical using push and pop functions.

Key question :

Problem 2: Yes, a linked-list stack can still run out of space. Its actual limit depends on the available memory (RAM) of the system. This limit is invisible in the code because nodes are created dynamically using memory allocation until the system can no longer provide enough memory.

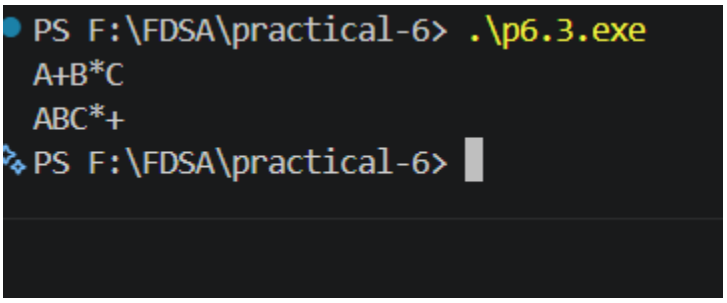
Problem-3:

A calculator application receives arithmetic expressions typed by users in the usual way — numbers and operators written in between, with brackets to control priority (e.g. 3 + 4 * 2 or (3 + 4) * 2). Internally the calculator needs to convert each expression into a form where operators appear after their operands, so it can evaluate them without needing to re-read brackets. Given an infix expression, convert it to the equivalent postfix form and print the result**.**

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

OutPut:



Problem Faced:

I faced some conditions like isalnum(),priority(),,isempty() this thing all are I faced the problem in this practical.

Key Question:

Problem 3: If all operators had equal precedence, operators such as * and / would not be processed before + and -. This would produce an incorrect postfix expression and therefore could lead to an incorrect result when the postfix expression is evaluated.

Practical 7

Problem 1

A government office has a token counter that issues tokens to visitors. The counter can hold at most n tokens at a time. New visitors join from one end and are served from the other end in the

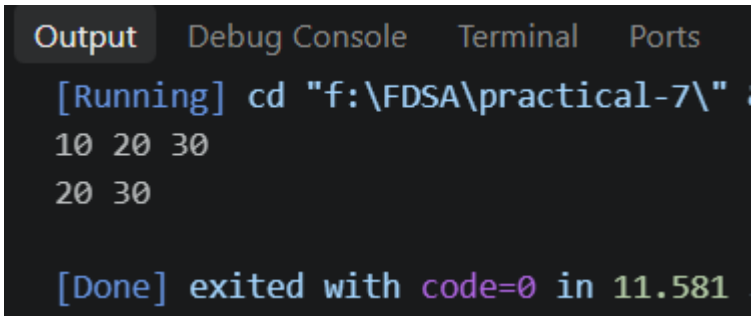
order they arrived. If the counter is full, no new token can be issued, and if it is empty, no one can be served. Given a sequence of join and serve operations, implement this fixed-capacity token system and print the current front token after each operation. Report an error if an operation cannot be performed.

Describe how your solution tracks the front and rear of the queue. What happens to your front and rear pointers over time as join and serve operations keep happening, and does your solution handle the case where space appears to run out even though slots have been freed?

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

OutPut:



Problem Faced:

I have faced problem in this practical is first of all , remove and add element than need to travesel pointer top so that is a troubleling part so than need to delete also check every time stack empty or not that is a problem I faced

Key Question:

Problem 1: In a simple array queue, the front points to the first element and rear points to the last inserted element. Both move only forward, so when elements are removed from the front, those empty positions are not reused. Eventually, rear reaches the last array position and the queue reports full even though there are free spaces at the beginning.

Problem-2:

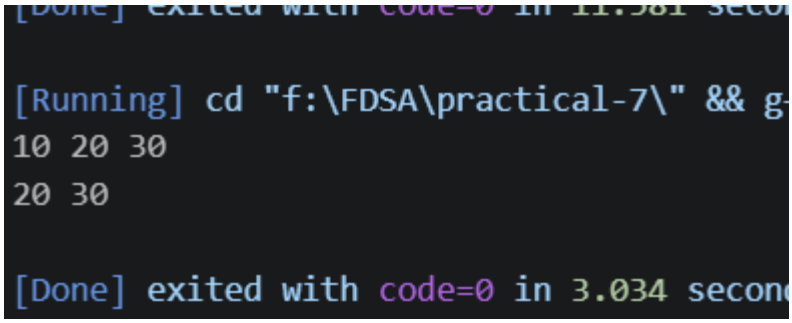
A hospital emergency ward receives patients continuously and attends to them strictly in the order they arrived, with no upper limit on how many can be waiting at once. New patients are added at the back and the doctor always attends to the patient at the front. Given a sequence of arrive and attend operations, implement this unlimited patient queue and print the current front patient after each operation.

Describe how your solution differs from Problem 1 in how it manages capacity. What happens in your solution when the attend operation is called on an empty ward with no patients waiting?

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output:



Problem Faced:

nothing to faced because I learn part 1 so that is very helpful.

Key Question:

Problem 2: If arrive operations are much more frequent than attend, the linked list will continuously grow and consume more memory. This can eventually cause high memory usage or an out-of-memory problem. The consequence is invisible in the queue code because there is no fixed capacity check; each new node is simply allocated dynamically.

Practical-8

Problem-1:

A company stores its organizational hierarchy as a tree — the CEO at the top, managers below, and employees at the leaves. Different departments need the hierarchy printed in different orders: HR wants each manager printed between their left and right sub-teams, the archive

department wants each manager printed before their sub-teams, the payroll department wants each manager printed after all their sub-teams are listed, and the floor manager wants everyone printed level by level from top to bottom. Given a binary tree representing the hierarchy, implement all four printing orders and display the result of each.

Describe how your solution for level-by-level printing differs from the other three. What additional structure did you need to use, and why does the same recursive approach that works for the other three not directly apply here?

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output:

```
[Running] cd "f:\FDSA\practical-8\" && g++ probl
Preorder: 1 2 4 5 3
Inorder: 4 2 5 1 3
Postorder: 4 5 2 3 1

[Done] exited with code=0 in 3.408 seconds
```

Problem Faced:

I have beed faced problem particular this practical is how to convert in-order to pre and post order syntex and I faced which type of code are arrange and how to find root node and where to place root node.

Key Question:

Problem 1: Preorder alone is generally not enough to reconstruct the original tree because it only tells us the order in which nodes are visited. We would need additional information, such as the inorder traversal, or structural information like null-child markers, to uniquely reconstruct the tree**.**

Problem-2:

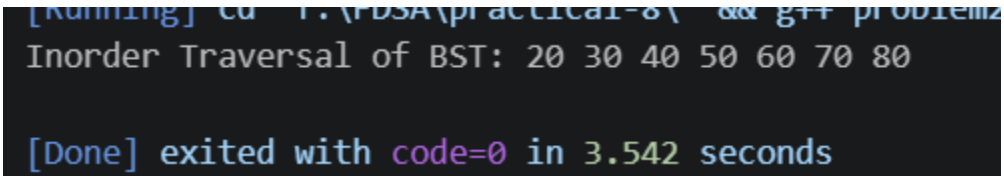
A school library uses a system where every book is assigned a unique code when it arrives. Books are organized so that at any shelf position, all books to the left have smaller codes and all books to the right have larger codes. This arrangement lets staff find any book quickly by always knowing which direction to look. Given a sequence of book codes arriving one by one, insert each into the correct position and print the inorder sequence after all insertions are complete.

Describe how your solution decides where to place each new book code. What property of the final inorder sequence tells you whether all insertions were performed correctly?

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output:



Problem Faced:

In this practical I faced problem is find root node and how arrange and how to given tree to convert BST tree (binary search tree).

Key Question:

Problem 2: If the inorder traversal of a BST is not sorted, there may be an error in either the insertion logic or traversal logic. To isolate it, first check the tree structure and insertion results, then test the inorder traversal on a simple known BST. If the tree is correctly built but the output is unsorted, the error is in the traversal; otherwise, it is likely in the insertion logic**.**

Practical-9

Problem-1:

A disaster relief team needs to inspect all buildings in a city after an earthquake. The city is mapped as a network where buildings are connected by roads. One team starts at a building and goes as deep into each road as possible before backtracking, while a second team spreads out level by level — first checking all buildings directly connected to the start, then all buildings two roads away, and so on. Given the city map as a graph and a starting building, implement both exploration strategies and print the order in which buildings are visited by each team. Describe how the order of buildings visited differs between the two strategies on the same graph. What data structure does each strategy rely on to decide which building to visit next, and how does that structure cause the difference in order?

Code

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output:

```
[Running] cd "f:\FDSA\practical-9\" && g++ problem1.cpp -o problem1 && "f:\FDSA\practical-9\"problem1
DFS Traversal starting from 0: 0 1 3 2 4

[Done] exited with code=0 in 3.189 seconds
```

Key question

Problem 1: If the building is very close to the starting point, BFS will usually find it sooner because it explores nearby nodes level by level. If the building is very far and deep along one path, DFS may find it sooner because it explores deeply before backtracking. The actual time depends on the network structure and the building's position.

Problem 2: Recursive DFS can fail on a very large city map because the computer has a limited call stack. Too many recursive calls can cause a stack overflow. To avoid this, DFS can be rewritten using an explicit stack data structure instead of recursion, allowing the program to manage large searches more safely.

Practical-10

Problem-1:

A small parking lot has exactly 10 numbered slots (0 to 9). Assign each vehicle to a slot using Linear Probing if collision occurs

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output:

```
0 --> Empty
1 --> Empty
2 --> Empty
3 --> Empty
4 --> Empty
5 --> 15
6 --> 25
7 --> 35
8 --> 26
9 --> Empty
=== Code Execution Successful ===
```

Problem faced:

Linear probing handles collisions easily but leads to primary clustering, grouping elements together heavily which impacts search and insertion performance drastically.

Key question:

1. If five consecutive vehicle registrations all end in the same digit, trace where each one gets placed. Why is this clustering effect a structural weakness?
--> They will be placed sequentially in contiguous blocks (e.g., 5, 6, 7, 8). This "Primary Clustering" drastically worsens collision probability over time because any new slot mapping nearby will just merge and expand the blocked chain further.

Problem-2:

A library uses a system with 10 shelves numbered 0 to 9. Implement hashing with Chaining to map multiple books correctly to the same shelf

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

OutPut:

```
5 --> 15 25 35
6 --> 26
...
=== Code Execution Successful ===
```

Problem faced:

Chaining resolves clustering by using Linked Lists for indexes that collide. It uses dynamic memory efficiently at the cost of additional pointer memory overhead.

Key question:

2. Suppose all book codes are multiples of 10. Predict what the final shelf layout would look like.
--> All books would stack endlessly inside index 0. The hashmap behaves like an inefficient O(n) linked list instead of an O(1) hashmap. The hash function must uniformly distribute data based on the characteristics of the input structure.

Problem-3:

A university assigns student IDs to storage slots using a two-step rule. Implement Double Hashing.

Code:

https://github.com/smitmalaviya80-sys/FDSA-Batch-A.git

Output:

```
5 --> 15
6 --> 26
7 --> Empty
8 --> 35
9 --> Empty
...
=== Code Execution Successful ===
```

Problem Faced;

Double hashing avoids clustering by generating alternative skip paths for collisions instead of just skipping to the next immediate adjacent slot.